

Практическая работа №4

Рефакторинг и тестирование .NET микросервисов

PracticalWork.Library

Филиппов Иван Ильич, группа 11-312

01

Цели, задачи и стек

Кратко: привести решение к чистым границам, обновить платформу и подтвердить качество тестами.

Архитектура

- проанализировать исходные зависимости
- отделить Library от Reports
- убрать лишние связи между слоями

Код и платформа

- перевести проекты на net10.0
- обновить пакеты и Dockerfiles
- добиться чистого restore/build

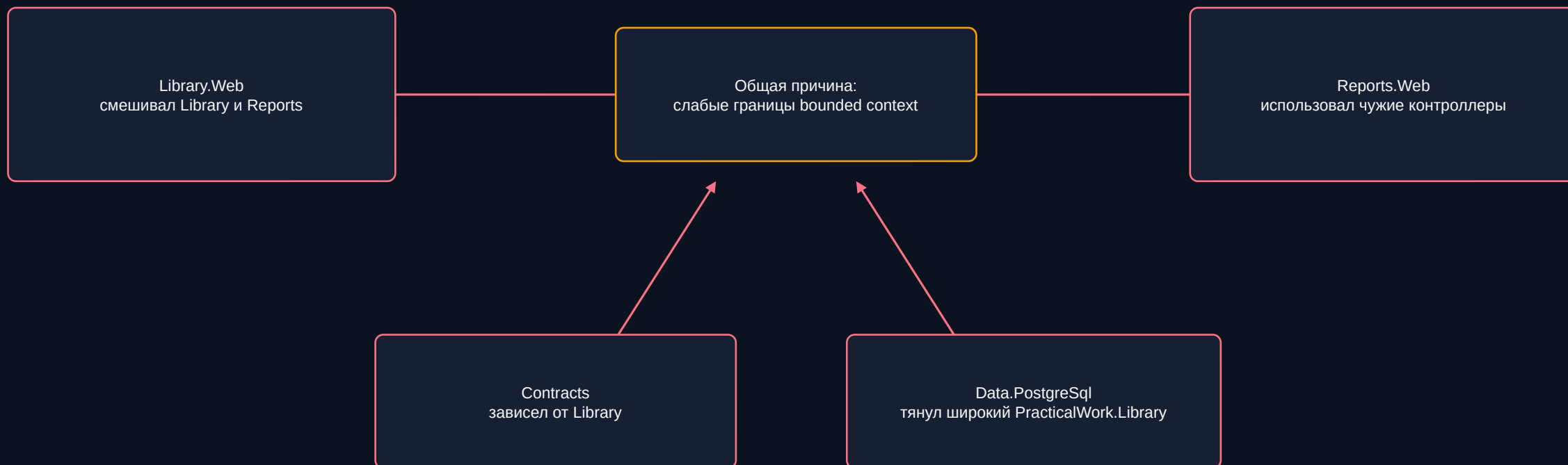
Качество

- покрыть бизнес-логику unit-тестами
- проверить Application/Domain coverage
- оформить README, Swagger и XML docs

Технология	Назначение	Статус
.NET	серверная платформа	10.0
PostgreSQL / Redis / MinIO	данные, кэш, файлы	через adapters
RabbitMQ / Hangfire / MailKit	сообщения, jobs, email	подключены
xUnit / Moq / coverlet	unit-тесты и coverage	70 тестов проходят

Что было проблемой в исходном решении

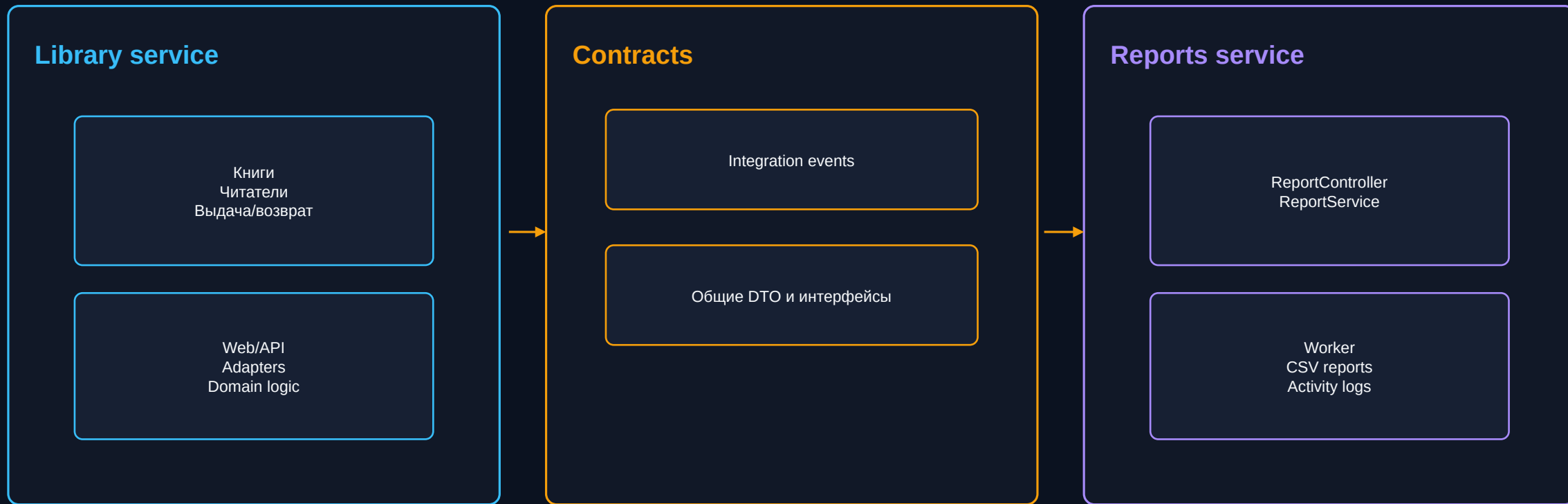
Основная проблема - сервисы знали слишком много друг о друге и тянули чужие runtime-сборки.



Вывод: точечные правки не решали проблему - требовалось перестроить зависимости вокруг нейтрального contract layer.

Целевая схема: чистые границы сервисов

Reports должен взаимодействовать с Library через контракты, а не через API-слой и доменные модели Library.



Результат: каждый микросервис имеет свой presentation и infrastructure слой, а общие точки взаимодействия вынесены в Contracts.

Ключевые архитектурные изменения

Смысл изменений - не просто переместить файлы, а убрать зависимость Reports от runtime Library.

Было	Стало	Эффект
Reports.* -> PracticalWork.Library	Reports.* -> PracticalWork.Library.Contracts	Reports не тянет runtime-сборку Library
Contracts зависел от Library	Contracts стал нейтральным слоем	контракты не знают домен Library
Reports.Web использовал Library.Controllers	Reports.Web получил свой ReportController	presentation-граница стала самостоятельной
Report-инфраструктура брала типы из Library	Data.Reports/MinIO/Cache/Broker работают через Contracts	инфраструктура Reports отвязана от Library domain

Что проверено в коде

- в Reports.Web и Reports.Worker нет ProjectReference на PracticalWork.Library.csproj
- report storage зависит от PracticalWork.Library.Contracts
- Reports.Web содержит собственные controllers, validators и MVC setup

Контроллер Reports: тоньше и самостоятельнее

ReportController перенесен в Reports.Web и больше не использует мапперы из Library.Controllers.

До

PracticalWork.Library.Controllers

```
using PracticalWork.Library.Abstractions.Services;  
using PracticalWork.Library.Controllers.Mappers.v1;
```

```
ReadSystemActivityLogs(request.ToGetActivityLogsRequestModel())
```

После

PracticalWork.Reports.Web.Controllers

```
using PracticalWork.Library.Contracts.Abstractions.Services;  
using PracticalWork.Library.Contracts.v1.Reports.Request;
```

```
_reportService.ReadSystemActivityLogs(request)  
_reportService.CreateReport(request)
```

Итог: Reports API публикуется через собственный Web-слой, а контроллер принимает request и передает его в contract service без лишних преобразований.

Репозиторий Reports переведен на Contracts

Data layer Reports больше не связан с доменными типами Library.

Было: широкая зависимость

```
using PracticalWork.Library.Abstractions.Storage;
using PracticalWork.Library.Models.ReportModels;
using PracticalWork.Library.Enums;
using PracticalWork.Library.Exceptions;

AddScoped<IReportRepository, ReportRepository>();
```

Стало: contract-only

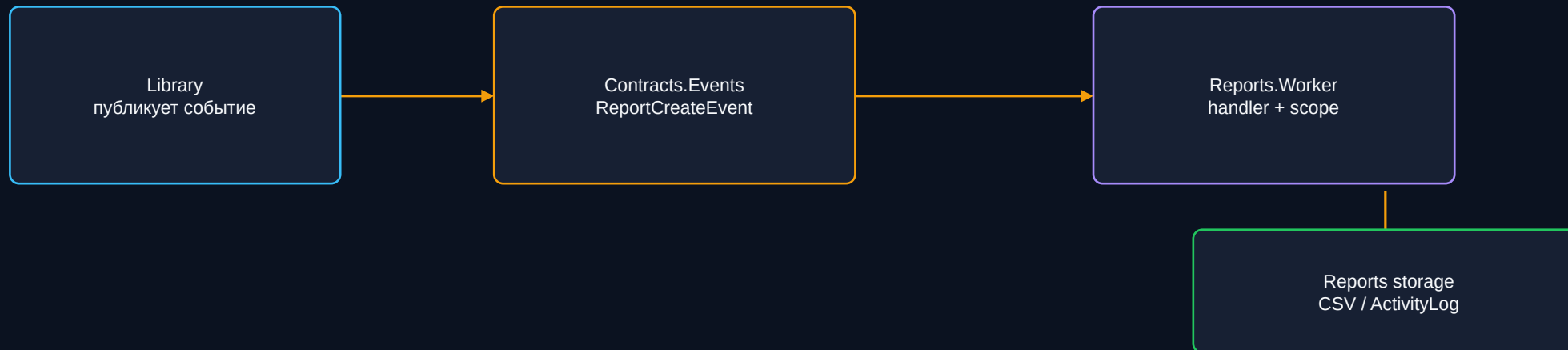
```
using PracticalWork.Library.Contracts.Abstractions.Storage;
using PracticalWork.Library.Contracts.Models.ReportModels;
using PracticalWork.Library.Contracts.Exceptions;

AddScoped<IReportRepository, ReportRepository>();
```

Элемент	Архитектурный смысл	Кодовый результат
ReportRepository	data layer Reports не зависит от домена Library	типы заменены на Contracts.*
AddReportsPostgreSqlStorage	регистрация report storage отдельно	IActivityLogRepository и IReportRepository из Contracts
ReportsDbContext	остается инфраструктурной деталью	не протекает в контроллеры

Worker и события: независимый фоновый контур

ReportConsumerWorker обрабатывает события через contract model и локальные модели worker.



Что изменилось

- событие живет в нейтральном contract layer
- worker не использует report модель из PracticalWorkLibrary
- результат генерации отчета хранится в локальной модели worker

Кодовый эффект

- обработка использует Contracts.Abstractions.MessageBroker
- worker создает scope и вызывает report handler
- фоновая логика отчетов отделена от Library domain

Структура unit-тестов

Тесты закрывают сервисы, контроллеры, validation, worker-сценарии и contracts.



Проект	Группы тестов	Что проверяется
PracticalWork.Library.Tests	Services, Models, Handlers	книги, читатели, выдача, архивация, уведомления, weekly report
PracticalWork.Reports.Tests	Controllers, Services, Validations, Worker, Contracts	Reports API, генерация отчетов, обработчики событий
TestCommon	StaticOptionsMonitor, TestTimeProvider	изоляция времени и options

Сценарии: success, invalid input, not found, conflicts и проверки вызовов зависимостей. Email/report тестируются без реальной отправки.

Покрытие Application / Domain

Покрытие измерялось отдельным фильтром, чтобы оценить именно прикладную и доменную логику.

80.0%

Application / Domain

874/1093

покрытых строк

100%

Reports.Web

100%

Reports.Worker

77.5%

Library

73.1%

Contracts

Как запускалось

Фильтр: Application/Domain
runsettings: coverage.app-domain.runsettings
HTML-отчет: artifacts/coverage-report/app-domain/index.html

Интерпретация

Покрытие не является самоцелью: оно показывает, какие части доменной и прикладной логики проверены, а где нужны дополнительные тесты.

Переход на .NET 10

Проекты обновлены до net10.0, пакеты выровнены, контейнерные сборки приведены к новой платформе.

12

проектов net10.0

10.0.201

SDK проверки

0 / 0

warnings / errors

10.0.0

EF Core, Npgsql, Microsoft.*

Этап	Что сделано	Результат
TargetFramework	все .csproj переведены с net8.0 на net10.0	артефакты в bin/Debug/net10.0
Central packages	Microsoft.*, EF Core, Npgsql выровнены на 10.x	совместимые версии в Directory.Packages.props
NU1903	лишние EF Tools/Design убраны из runtime library	restore/build без ошибок
Dockerfiles	web/worker/migrator обновлены под .NET 10	контейнерная сборка соответствует платформе

Подготовленные материалы

Документация описывает запуск, инфраструктуру, API и результаты инженерных изменений.

README.md / init.md

Структура решения, запуск окружения, инфраструктура, API routes и Docker Compose.

Swagger + XML docs

Публичное описание API для Library.Web и Reports.Web, включая комментарии к контрактам.

Coverage artifacts

HTML-отчеты по покрытию, сформированные через reportgenerator для выбранных фильтров.

Итог: материалы позволяют повторить сборку, проверить API, посмотреть покрытие и понять, зачем были выполнены архитектурные изменения.

Выводы по работе

В результате были улучшены границы микросервисов, обновлена платформа и подтверждено качество решения.

Архитектура и код

- Library и Reports разделены лучше
- Reports.* работают через Contracts
- controller/repository/worker показаны в before/after логике

Качество и платформа

- проекты переведены на .NET 10
- dotnet build без warnings/errors
- 70 unit-тестов проходят успешно

Покрытие и отчеты

- Application/Domain coverage = 80%
- HTML-отчеты сформированы через reportgenerator
- тесты помогают контролировать критичную логику

Документация

- подготовлен README
- настроены Swagger и XML documentation
- итоговые материалы оформлены для сдачи